

Model Compression and Efficiency Optimisation for Edge Deployment

Diploma Seminar 2 — Presentation 2
Results and Evaluation

Hubert Jaczynski

Warsaw, 2026

Warsaw University of Technology

Supervisor: PhD. Maria Ganzha

Thesis reminder

- The thesis studies **model compression and efficiency optimisation** for real-time object detection.
- The practical target is edge-oriented deployment, where accuracy alone is not enough.
- The central comparison is between detector families and compression variants under one benchmark workflow.
- This presentation focuses on the current results, not the full literature review.

Question behind the thesis: which detector and compression path gives the best accuracy–latency trade-off on constrained hardware?

Why this problem matters

- Object detectors are often evaluated mainly by COCO mAP, but deployment is constrained by more than accuracy.
- A detector may be accurate but unsuitable if it is too slow, too large, or too expensive to run continuously.
- Edge-like systems usually have:
 - strict latency and throughput limits,
 - limited GPU memory or accelerator memory,
 - device-specific runtime support,
 - power and thermal constraints.
- Therefore, the thesis treats detection as an **accuracy–latency–runtime** problem.

Research objective

Main objective

Identify which compression and efficiency optimisation techniques, or combinations of techniques, provide the best accuracy–latency–size trade-off for real-time object detection on constrained hardware.

- The work compares published COCO-trained detectors under controlled local measurements.
- It separates architecture choice from runtime/precision choice.
- It then prepares the ground for token reduction, pruning, and compression-order experiments.

Research questions

1. Which detector family gives the best accuracy–latency trade-off before compression?
2. How much speed can be gained from FP16 and TensorRT INT8 PTQ?
3. Do newer YOLO variants behave differently from YOLOv8 under the same precision changes?
4. Does a transformer-style detector need token-level compression to become competitive in latency?
5. Which compression sequence should be explored next: quantisation first, token reduction first, or pruning first?

Working hypotheses

- **H1:** INT8 PTQ should reduce latency more strongly than FP16, but with a measurable mAP drop.
- **H2:** the mAP drop from compression is architecture-dependent, not a fixed percentage.
- **H3:** RF-DETR should provide a strong accuracy anchor, but FP16 alone will not be enough for low-latency deployment.
- **H4:** newer YOLO variants should shift the baseline Pareto curve before any extra pruning or token reduction is applied.

Scope and assumptions

- Main dataset: COCO 2017 validation.
- Main hardware: NVIDIA GeForce RTX 3070 Ti with 8 GB VRAM.
- Main batch size: 1, because real-time detection is usually latency-sensitive.
- Compression currently evaluated: FP16 inference and INT8 PTQ for YOLO through TensorRT.
- Compression still planned: RF-DETR/Swin token reduction, RF-DETR INT8 export, light structured pruning.
- The results are preliminary thesis results, not final deployment claims for every edge device.

Model families in the benchmark

Family	Representative	Reason for inclusion
Detection transformer	RF-DETR Medium	High COCO accuracy and transformer-style detection pipeline.
Hierarchical ViT	Swin-T Mask R-CNN	Planned comparison for windowed/hierarchical attention.
YOLO baseline	YOLOv8s, YOLOv8m	Mature dense-detector baseline with strong tooling.
Newer YOLO	YOLO26s, YOLO26m	Newer Ultralytics family requested for the thesis comparison.

Design idea: compare models that represent different architecture choices, then apply comparable compression axes.

Compression axes

- **Precision reduction:** FP32 baseline, FP16 inference, INT8 post-training quantisation.
- **Token reduction:** planned for RF-DETR and Swin, where attention operates over visual tokens.
- **Structured pruning:** planned for YOLO-style models, where channel/filter pruning maps better to hardware.
- **Ordering:** later ablations will compare whether quantisation before pruning/token reduction is better than the reverse.

What changed since Presentation 1

- The benchmark workflow is now operational on the local RTX 3070 Ti.
- COCO 2017 validation data and annotations are available locally.
- RF-DETR Medium has completed FP32 and FP16 evaluation.
- YOLOv8s/m and YOLO26s/m have completed FP32, FP16, and TensorRT INT8 PTQ evaluation.
- Swin-T Mask R-CNN is still queued, but native Windows/OpenMMLab setup is the current blocker.

Main point: the first full evaluation already gives a clear accuracy–latency picture for transformer and YOLO-style detectors.

Evaluation protocol

Data

- Dataset: COCO 2017 validation split.
- RF-DETR evaluation: COCO bbox metrics on full val2017.
- YOLO evaluation: Ultralytics validation with COCO labels generated from the COCO JSON annotations.
- INT8 PTQ calibration: fixed 500-image COCO subset.

Measurements

- Accuracy: COCO mAP@0.5:0.95 and AP@0.5.
- Speed: median latency, batch size 1.
- Efficiency: throughput, GPU telemetry, model size where available.
- Hardware: NVIDIA GeForce RTX 3070 Ti, 8 GB VRAM.

The runner records one row per model/variant and preserves skipped rows, so missing components are visible instead of silently omitted.

Data preparation and transformations

- COCO annotations are stored in the original JSON format for pycocotools evaluation.
- For YOLO validation, the workflow converts COCO bounding boxes into YOLO-format text labels.
- INT8 PTQ uses a fixed calibration subset of 500 validation images.
- Generated labels, exports, raw metrics, and telemetry are separated from the thesis source files.

Why this matters: the same data source is used across frameworks, but each framework receives the format it expects.

How the metrics should be read

- **mAP@0.5:0.95:** main COCO detection-quality metric; higher is better.
- **AP@0.5:** easier overlap threshold; useful as a secondary quality check.
- **Median latency:** main deployment metric for batch size 1; lower is better.
- **Throughput:** images per second; useful, but less important than latency for live detection.
- **Power and memory telemetry:** early efficiency indicators, not yet final thesis claims.

Primary plot: mAP versus median latency, because it shows the deployment trade-off directly.

Hardware and runtime stack

Component	Current setup
GPU	NVIDIA GeForce RTX 3070 Ti, 8 GB VRAM
Driver / CUDA path	NVIDIA driver 595.97, PyTorch CUDA 13.0 build
Frameworks	PyTorch, Ultralytics, RF-DETR, pycocotools
Acceleration	TensorRT available for YOLO INT8 PTQ export
Telemetry	NVML sampling for power, utilisation, temperature, and memory
Blocked component	MMDetection/MMCV for Swin on native Windows

Interpretation: the results are local hardware measurements, so the ranking is more meaningful than FLOPs alone.

1. Run environment and dataset checks.
2. Build a manifest of model, variant, runtime, and batch-size jobs.
3. Run smoke tests first to catch broken labels, exports, or framework errors.
4. Run the overnight profile and write each result row immediately.
5. Store raw framework metrics and a summary CSV for later plots.
6. Preserve skipped rows with reasons, so the matrix remains auditable.

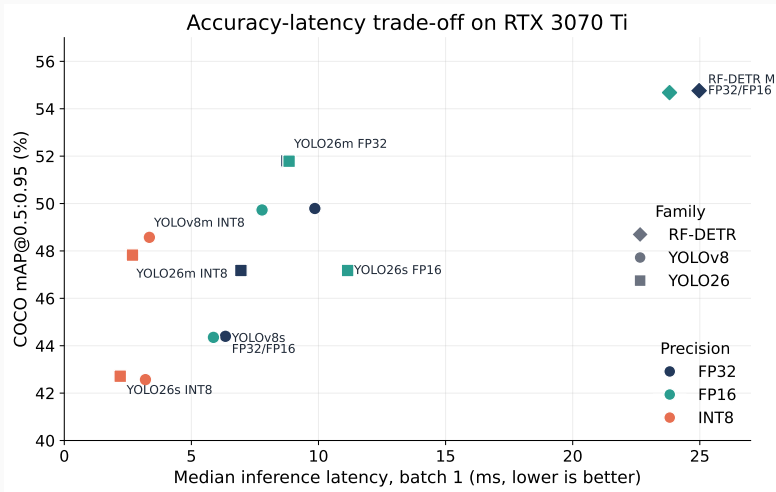
Practical result: a failed Swin or RF-DETR INT8 job does not erase the completed YOLO/RF-DETR evidence.

Experiment matrix status

Family	Model	FP32	FP16	INT8 PTQ
Detection transformer	RF-DETR Medium	<i>done</i>	<i>done</i>	<i>pending</i>
Hierarchical ViT	Swin-T Mask R-CNN	<i>blocked</i>	<i>blocked</i>	<i>blocked</i>
YOLO baseline	YOLOv8s, YOLOv8m	<i>done</i>	<i>done</i>	<i>done</i>
Newer YOLO variant	YOLO26s, YOLO26m	<i>done</i>	<i>done</i>	<i>done</i>

- Completed rows are sufficient for the first accuracy–latency frontier.
- Missing rows are methodological work items: Swin environment setup and RF-DETR ONNX/TensorRT export.
- Token reduction and pruning remain planned compression stages, after the precision baselines are stable.

Overall result: accuracy–latency frontier



Topline candidates

Model	Variant	mAP (%)	AP@0.5 (%)	Latency (ms)	Interpretation
RF-DETR M	FP32	54.76	73.63	24.97	Best accuracy
RF-DETR M	FP16	54.68	73.66	23.81	Same accuracy, lower power
YOLO26m	FP32	51.80	69.06	8.76	Strong YOLO baseline
YOLOv8m	INT8	48.57	65.25	3.35	Best INT8 stability
YOLO26m	INT8	47.82	65.98	2.68	Fast high-accuracy option
YOLO26s	INT8	42.71	59.39	2.20	Fastest measured detector

Reading: if accuracy is the priority, RF-DETR is still ahead. If deployment speed is the priority, TensorRT INT8 YOLO variants move into a much faster latency band.

RF-DETR evaluation

Variant	mAP (%)	AP@0.5 (%)	Latency (ms)	Throughput (img/s)	Avg. power (W)
FP32 PyTorch	54.76	73.63	24.97	40.04	136.93
FP16 PyTorch	54.68	73.66	23.81	42.00	91.21

- FP16 preserves detection quality almost exactly: only 0.08 percentage points of mAP difference.
- Latency improves only modestly, from 24.97 ms to 23.81 ms.
- Power telemetry improves much more strongly, which makes FP16 still relevant for deployment.

YOLO FP32 baselines: old vs newer variants

Model	mAP (%)	Latency (ms)	Params (M)	FLOPs (G)
YOLOv8s	44.40	6.34	11.2	28.6
YOLO26s	47.17	6.95	9.5	20.7
YOLOv8m	49.79	9.86	25.9	78.9
YOLO26m	51.80	8.76	20.4	68.2

- YOLO26m is the strongest YOLO baseline in this run: higher mAP and lower latency than YOLOv8m.
- YOLO26s improves accuracy over YOLOv8s with fewer parameters and lower reported FLOPs, but latency is slightly higher.
- This confirms that architecture choice changes the deployment trade-off before compression is applied.

Small YOLO models: YOLOv8s vs YOLO26s

Model	mAP (%)	Latency (ms)	Params (M)	FLOPs (G)
YOLOv8s FP32	44.40	6.34	11.2	28.6
YOLO26s FP32	47.17	6.95	9.5	20.7

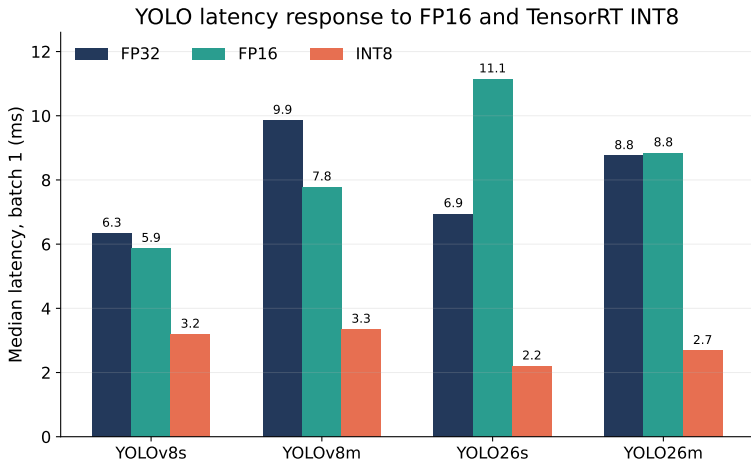
- YOLO26s improves accuracy by 2.77 mAP points over YOLOv8s.
- It does this with fewer parameters and lower reported FLOPs.
- The latency is slightly worse in this run, showing that FLOPs alone do not predict wall-clock speed.
- The small-model result is useful for future pruning because it starts from a compact baseline.

Medium YOLO models: YOLOv8m vs YOLO26m

Model	mAP (%)	Latency (ms)	Params (M)	FLOPs (G)
YOLOv8m FP32	49.79	9.86	25.9	78.9
YOLO26m FP32	51.80	8.76	20.4	68.2

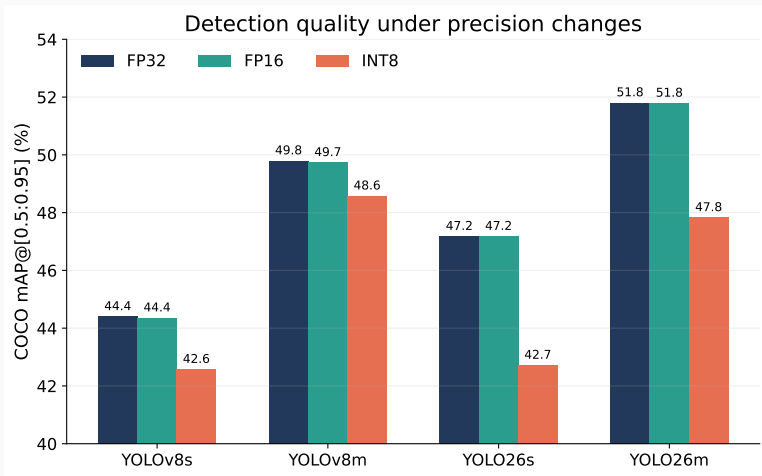
- YOLO26m dominates YOLOv8m in this local run: higher mAP and lower latency.
- The improvement is also parameter-efficient: 20.4M parameters versus 25.9M.
- This makes YOLO26m the strongest uncompressed YOLO candidate in the current matrix.
- It is also a good candidate for QAT or pruning experiments if INT8 PTQ remains too lossy.

FP16 is not automatically a free win



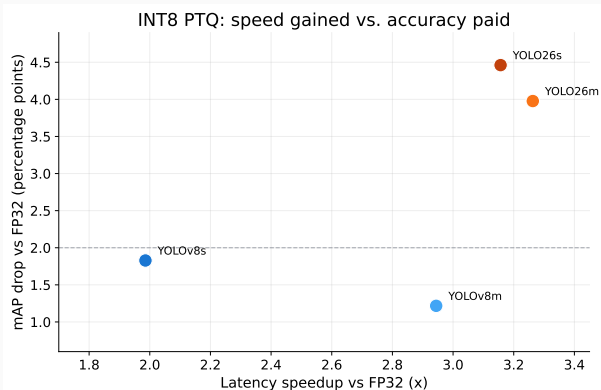
INT8 uses TensorRT; FP32 and FP16 use the PyTorch/Ultralytics path.

Accuracy under precision changes



FP16 preserves mAP; INT8 PTQ has an architecture-dependent accuracy cost.

INT8 PTQ trade-off



INT8 speedup versus mAP loss relative to each FP32 baseline.

Model	Speedup (x)	mAP loss (pp)
YOLOv8s	1.99	1.83
YOLOv8m	2.94	1.22
YOLO26s	3.16	4.46
YOLO26m	3.26	3.98

Interpretation: YOLOv8m is the most robust to PTQ, while YOLO26 gains more latency but pays a larger mAP cost.

Best candidates by deployment priority

Priority	Candidate	Reason
Maximum accuracy	RF-DETR M FP16 54.68 mAP, 23.81 ms	Highest quality; FP16 reduces power with almost no accuracy loss.
Balanced YOLO	YOLO26m FP32 51.80 mAP, 8.76 ms	Best uncompressed YOLO trade-off in the current run.
Robust INT8	YOLOv8m INT8 48.57 mAP, 3.35 ms	Lowest mAP loss from PTQ among the YOLO INT8 variants.
Fast INT8	YOLO26m INT8 47.82 mAP, 2.68 ms	Strong speed with still competitive AP@0.5.
Fastest measured	YOLO26s INT8 42.71 mAP, 2.20 ms	Lowest latency, but the accuracy cost is larger.

Thesis implication: there is no single winner; the best choice depends on whether the target optimises for accuracy, speed, or robustness to quantisation.

Evaluation against the research hypotheses

- **Precision compression improves deployment speed.**

Confirmed for TensorRT INT8 YOLO: roughly 2.0x–3.3x lower latency with measurable but bounded mAP loss.

- **Architecture affects compression behaviour.**

Confirmed: YOLOv8m loses only 1.22 pp mAP under INT8, while YOLO26m loses 3.98 pp despite being faster.

- **Transformer-style detectors need token/runtime compression, not only FP16.**

Partially supported: RF-DETR has the best accuracy, but FP16 alone does not change latency enough.

- **Ordering effects and pruning/token reduction still need experimental evidence.**

Not yet tested; this is the next ablation stage after the baseline precision grid.

Current limitations

- Swin-T Mask R-CNN is not yet evaluated because MMDetection/MMCV setup is unstable on native Windows with the current Python/CUDA stack.
- RF-DETR INT8 PTQ is queued, but requires an ONNX/TensorRT export path before it can be measured fairly.
- INT8 results use post-training quantisation with a 500-image calibration subset; QAT may recover accuracy, especially for YOLO26.
- Power and energy telemetry are useful early indicators, but final claims should be based on repeated runs after the workflow metadata fix.
- The RTX 3070 Ti is the current target hardware; final edge-device claims should distinguish GPU-local results from embedded deployment.

Next experiments

1. Re-run the overnight profile after the image-count metadata fix and freeze the curated result table.
2. Complete Swin-T evaluation in a compatible WSL2/Linux OpenMMLab environment.
3. Add RF-DETR ONNX/TensorRT export, then test INT8 PTQ.
4. Implement one conservative token-reduction setting for RF-DETR/Swin and compare actual latency, not only FLOPs.
5. Add one structured-pruning YOLO ablation and then test pruning + INT8 ordering.
6. Optional robustness check: blur, noise, and JPEG degradation for FP32 versus INT8 detectors.

The preliminary results already support the thesis direction: compression is useful, but the best method depends strongly on architecture and runtime.

- RF-DETR currently defines the accuracy ceiling: 54.8 mAP at about 25 ms.
- YOLO26m is the strongest dense-detector baseline before compression.
- TensorRT INT8 gives the largest deployment-speed gains, especially for YOLO, but the accuracy cost is architecture-dependent.